List of common time complexities along with their definitions:

1. **O(1) - Constant Time:**
    - The execution time is constant and does not change with the size of the input.
    - Example: Accessing an element in an array by index.
2. **O(log n) - Logarithmic Time:**
    - The execution time grows logarithmically as the size of the input increases.
    - Example: Binary search in a sorted array.
3. **O(n) - Linear Time:**
    - The execution time grows linearly with the size of the input.
    - Example: Iterating through all elements in an array.
4. **O(n log n) - Linearithmic Time:**
    - The execution time grows in proportion to n log n.
    - Example: Efficient sorting algorithms like Merge Sort or Quick Sort.
5. **O(n^2) - Quadratic Time:**
    - The execution time grows proportionally to the square of the size of the input.
    - Example: Bubble Sort or Insertion Sort.
6. **O(n^3) - Cubic Time:**
    - The execution time grows proportionally to the cube of the size of the input.
    - Example: Algorithms with three nested loops.
7. **O(2^n) - Exponential Time:**
    - The execution time grows exponentially with the size of the input.
    - Example: Recursive algorithms for problems like the Traveling Salesman Problem (TSP).
8. **O(n!) - Factorial Time:**
    - The execution time grows factorially with the size of the input.
    - Example: Generating all permutations of a set.

The best time complexity generally depends on the context of the problem and the requirements of the application. However, in general terms:

1. **O(1) - Constant Time** is considered the best in terms of efficiency because the execution time does not increase with the size of the input. Algorithms with O(1) time complexity are the most efficient, as their performance remains constant regardless of the input size.
2. **O(log n) - Logarithmic Time** is also very efficient, especially for operations like search in a sorted dataset. It's significantly better than linear or polynomial time complexities for large inputs.
3. **O(n) - Linear Time** is often acceptable for many practical applications, particularly when the input size is not excessively large.

4. **O(n log n) - Linearithmic Time** is very efficient for sorting algorithms and is often the best you can achieve for comparison-based sorting.

For most practical applications, aiming for O(1) or O(log n) time complexity is ideal. However, if O(1) or O(log n) is not achievable, you should consider the trade-offs and choose an algorithm with the best time complexity suitable for your specific needs.

Here are some common data structures that can offer O(1) time complexity for certain operations:

1. **Array:**
    - **Access by Index:** Directly accessing an element using its index is O(1).
2. **Hash Table (or Hash Map):**
    - **Insertions, Deletions, and Lookups:** Average-case time complexity for these operations is O(1), assuming a good hash function and low load factor. However, in the worst case (due to collisions), these operations can degrade to O(n).
3. **Stack (using an array):**
    - **Push and Pop Operations:** Both operations are O(1) when implemented using a dynamic array.
4. **Queue (using an array or linked list):**
    - **Enqueue and Dequeue Operations:** For a queue implemented with a circular buffer (array) or doubly linked list, both operations are O(1).
5. **Linked List (for head operations):**
    - **Insert at Head or Remove from Head:** Both operations are O(1) in a singly or doubly linked list.
6. **Deque (Double-ended Queue):**
    - **Add or Remove from Both Ends:** Operations like adding or removing elements from the front or back are O(1) when implemented using a doubly linked list or a circular buffer.

Each of these data structures provides O(1) time complexity for specific operations, making them highly efficient for those tasks. However, it's important to consider the overall efficiency of a data structure in the context of the operations you need to perform and the trade-offs involved.

Here are some data structures and algorithms that typically offer O(log n) time complexity for certain operations:

1. **Binary Search Tree (BST):**
    - **Search, Insertion, and Deletion:** For balanced BSTs like AVL Trees or Red-Black Trees, these operations are O(log n).
2. **Binary Search:**

    - **Search in a Sorted Array:** Finding an element in a sorted array using binary search is O(log n).
3. **Heap (Priority Queue):**
    - **Insertions and Deletions (Extract Min/Max):** In a binary heap, both operations are O(log n).
4. **B-Trees and B+ Trees:**
    - **Search, Insertion, and Deletion:** In B-Trees (used in databases) and B+ Trees (used in file systems), these operations are O(log n), where n is the number of keys.
5. **Segment Tree:**
    - **Query and Update Operations:** Both query and update operations in a segment tree are O(log n).
6. **Fenwick Tree (Binary Indexed Tree):**
    - **Update and Query Operations:** Both operations in a Fenwick Tree are O(log n).
7. **Balanced Tree Variants (like Splay Trees):**
    - **Search, Insertion, and Deletion:** In splay trees (a type of self-adjusting BST), these operations have an amortized time complexity of O(log n).

These data structures and algorithms are useful when you need efficient operations on dynamic sets or sorted collections, balancing fast access times with efficient updates.

Here are some data structures and algorithms that typically offer O(n) time complexity for certain operations:

1. **Array:**
    - **Traversal:** Iterating through all elements of an array is O(n).
2. **Linked List:**
    - **Traversal:** Traversing a singly or doubly linked list to visit each node is O(n).
    - **Search:** Searching for an element in an unsorted linked list is O(n).
3. **Hash Table (Worst Case):**
    - **Search, Insert, and Delete:** In the worst case, these operations can degrade to O(n) if many hash collisions occur.
4. **Queue (using an array):**
    - **Traversal:** Iterating through all elements in a queue implemented with an array is O(n).
5. **Stack (using an array):**
    - **Traversal:** Iterating through all elements in a stack implemented with an array is O(n).
6. **Counting Sort:**
    - **Sorting:** Counting sort has O(n) time complexity, assuming the range of elements is not significantly larger than n.
7. **Radix Sort:**

- **Sorting:** Radix sort has O(n) time complexity when the number of digits in the largest number (or the range of digits) is constant.

8. **Breadth-First Search (BFS) on a Graph:**
    - **Traversal:** In an unweighted graph, BFS will visit each node and edge once, making the time complexity O(n + e), where e is the number of edges. In terms of nodes alone, it's O(n).
9. **Depth-First Search (DFS) on a Graph:**
    - **Traversal:** Similar to BFS, DFS will visit each node and edge once, making the time complexity O(n + e). For traversal based solely on nodes, it's O(n).

These data structures and algorithms are often used when you need to process or analyze every element in a collection, and their linear time complexity ensures that operations scale directly with the size of the input.

Here are some data structures and algorithms that typically offer O(n log n) time complexity:

1. **Merge Sort:**
    - **Sorting:** Merge sort divides the array into halves, sorts each half, and then merges them. This process results in an O(n log n) time complexity.
2. **Heap Sort:**
    - **Sorting:** Heap sort involves building a heap from the elements and then repeatedly extracting the maximum (or minimum) to sort the array. This has a time complexity of O(n log n).
3. **Quick Sort (Average Case):**
    - **Sorting:** Quick sort, on average, has a time complexity of O(n log n). However, in the worst case (when the pivot choices are poor), it can degrade to O(n^2).
4. **Balanced Tree Operations (e.g., AVL Tree or Red-Black Tree):**
    - **Insertion, Deletion, and Search:** For operations in balanced binary search trees, such as AVL Trees or Red-Black Trees, the time complexity for balancing operations is O(log n), but for bulk operations involving multiple elements, it can be O(n log n).
5. **Balanced Search Trees (e.g., B-Trees, B+ Trees):**
    - **Insertion, Deletion, and Search:** In balanced search trees like B-Trees or B+ Trees, where the number of keys can be large, operations are generally O(log n), but building or processing the entire structure can be O(n log n).
6. **Sorting Algorithms Using Divide and Conquer:**
    - **Algorithms like Timsort:** Timsort (used in Python’s built-in sort function) combines merge sort and insertion sort and has an average time complexity of O(n log n).
7. **FFT (Fast Fourier Transform):**
    - **Transformations:** Computing the discrete Fourier transform using the Cooley-Tukey algorithm is O(n log n).

These algorithms and data structures are efficient for sorting and processing tasks where a combination of linear and logarithmic operations is required, making them suitable for a wide range of applications.

Here is the example of bubble sort and how to find the time complexities for this algorithm

To determine how many comparisons occur in the `bubble_sort_array_in_ascending_order` function, we need to focus on the number of iterations of the inner loop, which is where the comparisons take place.

Breakdown of Comparisons:

In Bubble Sort, for each pass of the outer loop, the inner loop performs a decreasing number of comparisons. Specifically:

- On the first pass (i = 0), the inner loop compares elements from `j = 0` to `j = n-2`, resulting in **(n-1) comparisons**.
- On the second pass (i = 1), the inner loop compares elements from `j = 0` to `j = n-3`, resulting in **(n-2) comparisons**.
- On the third pass (i = 2), the inner loop compares elements from `j = 0` to `j = n-4`, resulting in **(n-3) comparisons**.
- This continues until the last pass where **1 comparison** is made.

```
def bubble_sort_array_in_ascending_order(arr):
n = len(arr) # Get the length of the array
# Traverse through all elements in the array
for i in range(n):
# Traverse the array from 0 to n-i-1
# The last i elements are already sorted
for j in range(0, n-i-1):
if arr[j] > arr[j+1]: # Swap if the element found is greater than the next element
arr[j], arr[j+1] = arr[j+1], arr[j] # Swap the elements

# Define the array to be sorted
my_array = [64, 34, 25, 12, 22, 11, 90, 5]

# Apply the bubble sort function
bubble_sort_array_in_ascending_order(my_array)

# Print the sorted array
print("Sorted array:", my_array)
```

Total Number of Comparisons:

The total number of comparisons is the sum of all these comparisons:

$(n-1)+(n-2)+(n-3)+\ldots+1$

This is an arithmetic series. The formula for the sum of the first k natural numbers is:

$S=2k(k+1)$

For this case, $k=n-1$, so the total number of comparisons is:

$2(n-1)\cdot n$

This is the exact number of comparisons made in the bubble sort.

Example with an Array of Length 8:

If the array has $n=8$ elements:

$2(8-1)\cdot 8=27\cdot 8=28$

So, in this case, the function will make **28 comparisons**.

General Case:

For an array of size n, the number of comparisons is:

$2n(n-1)$

This number grows quadratically with the size of the array, which aligns with the time complexity of $\mathbf{O(n^2)}$.